Aumentación de Datos con LLMs

Filtrado Geométrico para Clasificación de Texto en Pocos Ejemplos

UAI | FACULTAD DE INGENIERÍA Y CIENCIAS

Benjamín Schindler

1 de junio de 2026

Facultad de Ingeniería y Ciencias
Universidad Adolfo Ibáñez

Profesor guía

Dr. Gonzalo A. Ruz

Grados a optar

Master of Science in Data Science
Ingeniería Civil Informática
Ingeniería Civil Industrial

Comité de defensa

Dr. Luis Aburto
Dr. Rolando De La Cruz
Dr. Pablo Henríquez
Dr. Andrés Peters

Hoja de ruta

1. **Introducción** — desbalance, voces que se callan y por qué las clases raras suelen ser las que más importan.
2. **Problema** — escasez de datos, el marco clásico del re-muestreo y la pregunta central.
3. **Estado del arte** — tres familias de aumentación, tres deficiencias persistentes y una década de evolución.
4. **Hipótesis y objetivos** — qué afirmamos y cómo se va a verificar.
5. **Metodología** — cinco filtros geométricos, ponderación suave y un pipeline desacoplado del LLM.
6. **Resultados** — 8.000+ configuraciones, +2.25 pp sobre SMOTE y un hallazgo contra-intuitivo.
7. **Discusión** — utilidad práctica, limitaciones honestas y trabajo futuro.

1.
Introducción

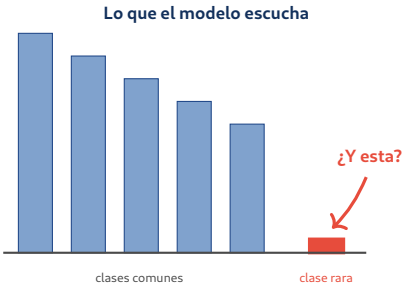
1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

¿Estamos escuchando todas las voces?

En la práctica, los datos **más importantes** suelen ser los **menos frecuentes**. La realidad rara vez se reparte por partes iguales:

- **Un** correo de phishing con un esquema nunca antes visto.
- Notas clínicas con síntomas raros: **un puñado** de menciones.
- **Un** mensaje de auxilio entre miles de consultas rutinarias.
- Discurso de odio en lenguas minorizadas: **decenas** de ejemplos.

Lo común tapa lo raro.
¿Y cómo afecta esto a los modelos de predicción?



¿Cómo aprende un modelo lo que apenas escucha?

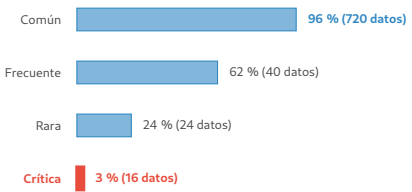
2.
Problema

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

Por qué los clasificadores se inclinan hacia las clases mayoritarias

Un modelo puede ser **correcto en el 95 % de los casos** y, aun así, **fallar en el 100 % de los que importan**.

Recall por clase en un dataset desbalanceado



Precisión global
95 %

Recall en clase crítica
0 %

El mismo modelo.
Dos verdades.

La voz que el modelo **no escuchó**
suele cargar con el mayor **costo humano**.

¿Cómo le enseñamos a un modelo a reconocer aquello que casi no ha visto?

¿Qué se hace para este desbalance?

La estadística clásica enmarca la escasez desde hace décadas como un problema de re-muestreo. Ambas familias caben aquí.

Undersampling

Reduce la clase mayoritaria.

Ejemplos: muestreo aleatorio, NearMiss, Tomek Links.

✓ Más rápido al entrenar.
✗ Pierde información. Inviable en few-shot.

Oversampling

Aumenta la clase minoritaria.

Ejemplos: duplicación, **SMOTE**, ADASYN, EDA, **LLMs**.

✓ Conserva todos los datos.
✗ Riesgo de overfitting, ruido o divergencia.

Dentro del **oversampling** existen métodos de **interpolación** y de **generación con IA generativa**.

¿Qué métodos existen, cómo funcionan y funcionan siempre?

3.
Estado del arte

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

Familia 1 — SMOTE y ADASYN: interpolación geométrica

SMOTE [Chawla et al., 2002] y **ADASYN** [He et al., 2008] son los métodos canónicos de oversampling.

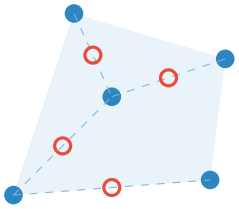
Idea:

- Para cada muestra minoritaria x_i, encontrar sus k vecinos más cercanos de la misma clase.
- Generar muestras nuevas sobre los segmentos:

$$x_{new} = x_i + \lambda\,(x_j - x_i), \quad \lambda \in [0, 1]$$

Diferencia clave:

- ADASYN pesa más las muestras difíciles (rodeadas de otra clase).



• reales ○ sintéticos

Interpolación entre k-NN

✓ Geometría controlada. ✓ Garantías formales.

✗ Genera vectores, no texto. ✗ Solo rellena lo conocido.

Geometría como control: pero la salida es un vector, no una oración.

Familia 2 — Aumentación textual ligera

Entre la interpolación geométrica y la generación libre hay un puente histórico: métodos que producen texto con modelos pequeños y operaciones controladas.

EDA

[Wei & Zou, 2019]

4 ops a nivel **palabra**:
sinónimo, inserción, swap, deleción.

✓ Muy barato.
✗ Ruido en oraciones cortas.

Traducción inversa

[Sennrich 2016; Edunov 2018]

EN → otro idioma → EN.
Paráfrasis a nivel **oración**.

✓ Cambia sintaxis.
✗ Pierde matices.

BERT contextual

[Kobayashi 2018; Wu 2019]

Enmascarar tokens; BERT predice alternativas plausibles.

✓ Contexto semántico.
✗ Solo sustitución 1:1.

Todos producen **texto**, pero con un **techo de creatividad bajo**: paráfrasis cercanas al original. Los LLMs rompen ese techo — y por eso necesitan otro tipo de control.

Familia 3 — LLMs: generación de texto bajo demanda

Pedirle a un LLM (GPT, Gemini, Claude…) que genere ejemplos nuevos para cada clase.

Receta típica [Whitehouse et al., 2023]:

1. Prompt con descripción de clase y ejemplos reales.
2. Pedir N generaciones nuevas.
3. Añadirlas al entrenamiento.



Trabajos representativos:

- Suhaeni & Yong (2024): +12,7 % acc. en minoritarias.
- ImbLLM (2024): control de desbalance vía prompting.
- Cegin et al. (2025): diversidad guiada por LLM.

✓ Texto coherente. ✓ Diversidad lingüística.

✗ Sin garantías geométricas. ✗ Ruido y solapamiento.

Texto sin control: los vectores caen donde quieran.

Familia 3.5 — Fine-tuning para generación sintética

Una rama más reciente: ajustar el LLM sobre la propia tarea antes de generar.

Idea: fine-tunear un LLM preentrenado (GPT-2, LLaMA, T5…) con los pocos ejemplos disponibles, y luego generar muestras específicas del dominio.



Trabajos representativos:

- Cloutier & Japkowicz (2023): fine-tuning generativo supera al re-muestreo tradicional en multiclase desbalanceada.
- ImbLLM (Nguyen et al., 2024): fine-tuning **solo en muestras minoritarias**; F1 $>$ 0.8.
- SentiGEN (Sundararajan, 2023): T5 fine-tuned + validador XLNet.

✓ Distribución alineada con la tarea.

✗ Requiere cómputo extra; riesgo de overfitting con n pequeño.

Más alineamiento, más costo: distinto eje de control al filtrado posterior.

El dilema: SMOTE vs. LLMs lado a lado

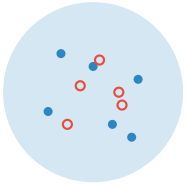
SMOTE / interpolación

✓ Geometría controlada

✓ Garantías formales

✗ Vectores sin texto coherente

✗ Solo rellena lo conocido



Interpolación lineal

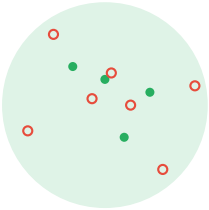
LLM (Gemini, GPT, Claude…)

✓ Texto fluido y coherente

✓ Diversidad lingüística

✗ Geometría descontrolada

✗ Ruido y solapamiento entre clases



Generación abierta

¿Podemos quedarnos con lo mejor de ambos mundos?

Tres deficiencias en los métodos actuales

1. Sin validación

Las muestras del LLM se aceptan **todas**, sin verificar si caen en la región correcta del espacio.

Texto gramatical $\neq$ texto en la zona correcta.

2. Peso uniforme

Todas las muestras pesan igual en el clasificador, ignorando la **calidad geométrica** de cada una.

Una en zona densa = una en zona dispersa.

3. Un solo dataset

Casi todos los estudios validan en **un único** corpus o tarea.

¿Generaliza el filtrado entre dominios y tareas?

La señal geométrica está disponible y nadie la está usando.

El gap metodológico: filtrar la salida del LLM

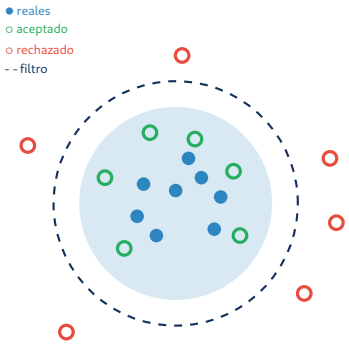
Las deficiencias anteriores apuntan a una intervención conceptualmente simple, no estudiada sistemáticamente.

La idea:

1. Generar N muestras con el LLM (como hoy).
2. Proyectarlas al espacio de embeddings.
3. Medir qué tan cerca caen del **cluster real** de su clase.
4. Rechazar las que se alejan demasiado.

¿Por qué tiene sentido?

- La geometría ya está ahí: usamos embeddings para el clasificador.
- Podemos usar **cualquier API de LLM SOTA** (Gemini, GPT, Claude…).



La señal geométrica ya existe — falta usarla para decidir qué entra al entrenamiento.

4.
Hipótesis y objetivos

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

Hipótesis y objetivos

H1 — Hipótesis principal

El filtrado geométrico de muestras LLM en el espacio de embeddings produce mejoras **estadísticamente significativas** respecto a SMOTE, en clasificación con pocos ejemplos por clase.

Objetivo general

Desarrollar y validar un sistema de filtrado geométrico que opere **exclusivamente en el espacio de embeddings**, **desacoplado del LLM generativo**, y mejore clasificadores de texto en escenarios de pocos ejemplos por clase.

Sub-hipótesis verificables

H1a Filtrado por distancia $>$ sin filtrar.
H1b Ponderación suave $\geq$ filtrado binario.

Objetivos específicos

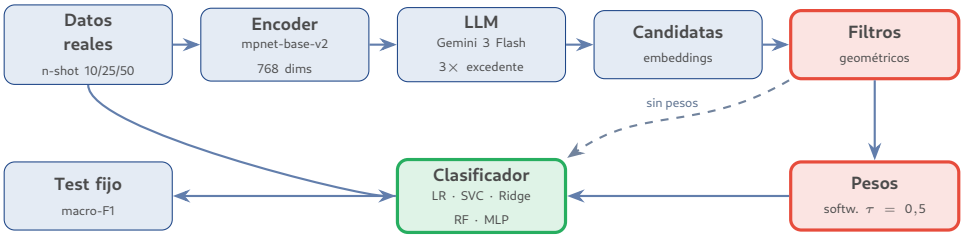
1. Diseñar el sistema de filtros con ponderación suave.
2. Comparar contra **9 métodos** (clásicos + modernos).

Criterio de aceptación: $\Delta > 0$, $p_{\text{Bonf}} < 0{,}05$, win-rate $> 50\,\%$.

5.
Metodología

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

Pipeline: del texto real al clasificador entrenado



Filtrado y ponderación: dos contribuciones desacopladas del LLM.

Protocolo experimental: 8.000+ configuraciones

Datos y modelos:

- **13 datasets** en 9 dominios.
- n**-shot:** 10 / 25 / 50 ejemplos por clase.
- **5 clasificadores:** LR, SVC, Ridge, RF, MLP.
- **10 métodos** de aumentación.
- **5 LLMs** (Gemini, GPT-5, Claude, Kimi, GLM).
- **4 modelos** de embeddings.
- **5 semillas:** 42, 123, 456, 789, 1011.

Protocolo estadístico:

- Métrica: **macro F1**.
- ΔF1 vs. SMOTE (puntos porcentuales).
- t**-pareada** con corrección de Bonferroni.
- Bootstrap CI 95 % (10 000 remuestreos).
- d de Cohen como tamaño de efecto.
- Win-rate como medida de consistencia.

Volumen

8.000+ configuraciones

Métrica de evaluación: Macro F1

Por qué no accuracy: con clases desbalanceadas, predecir siempre la clase mayoritaria puede dar 95 % de accuracy y 0 % en la clase rara.

Macro F1 promedia el F1 de cada clase con peso igual, sin importar su tamaño:

F1 por clase c:

$$\mathrm{F1}_c = 2 \cdot \frac{\mathrm{P}_c \cdot \mathrm{R}_c}{\mathrm{P}_c + \mathrm{R}_c}$$

Macro F1 sobre C clases:

$$\text{macro-F1} = \frac{1}{C} \sum_{c=1}^{C} \mathrm{F1}_c$$

donde $\mathrm{P}_c = \dfrac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c}$ y $\mathrm{R}_c = \dfrac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}$.

Propiedades clave

- Penaliza errores en **clases raras** igual que en clases grandes.
- Rango $[0, 1]$; $1 =$ perfecto.
- Detecta cuando el modelo ignora clases minoritarias.

Reporte de resultados

- $\Delta\mathrm{F1}$: diferencia frente a SMOTE (puntos porcentuales).
- Test fijo, mismo split en todas las configuraciones.

Macro F1: la métrica honesta cuando hay clases raras.

Conjuntos de datos: 13 corpus en 9 dominios

Conjunto	Dominio	Clases	n-shots	Test
Clasificación de texto — experimento principal				
`sms_spam`	Detección de spam	2	10, 25, 50	1.394
`hate_speech_davidson`	Discurso de odio	3	10, 25, 50	4.948
`20newsgroups`	Noticias (4 cat.)	4	10, 25, 50	7.532
`ag_news`	Noticias (4 cat.)	4	10, 25, 50	7.600
`emotion`	Emociones	6	10, 25, 50	2.000
`dbpedia14`	Clasificación de temas	14	10, 25, 50	5.000
`20newsgroups_20class`	Noticias (20 cat.)	20	10, 25, 50	7.532
Adicionales — escalabilidad por número de clases				
`trec6`	Preguntas	6	10, 25, 50	500
`banking77`	Intenciones bancarias	77	10, 25, 50	3.080
`clinc150`	Intenciones de diálogo	150	10, 25, 50	4.500
Reconocimiento de entidades nombradas (NER)				
`MultiNERD`	Entidades generales	5 tipos	10, 25, 50	10.000
`WikiANN`	Entidades Wikipedia	3 tipos	10, 25, 50	10.000
`Few-NERD`	Entidades detalladas	8 tipos	10, 25, 50	10.000

n-shot estricto: 10 / 25 / 50 ejemplos por clase. Rango de complejidad: de binario (*`sms_spam`*) a 150 clases (*`clinc150`*).

¿Qué es NER y por qué la usamos?

NER (Named Entity Recognition)

Tarea de NLP que **detecta nombres propios dentro del texto y los clasifica por tipo**: personas, lugares, organizaciones, fechas, etc.

Ejemplo: «Llamé a María en Madrid el 2 de marzo .» ↑PER ↑LOC ↑DATE

¿Por qué la usamos como prueba?

- Construimos el filtro para **clasificación**.
- NER es una tarea NLP **distinta**: en vez de etiquetar la oración entera, se etiquetan palabras dentro.
- Si el mismo filtro funciona en NER **sin cambiarlo** ⇒ el filtro no depende de la tarea.

3 corpus, distinta complejidad

- `WikiANN` — 3 tipos (PER, ORG, LOC). **Fácil**.
- `MultiNERD` — 5 tipos (+ DATE, MISC). Medio.
- `Few-NERD` — 8 tipos jerárquicos. **Difícil**.

Es la prueba más fuerte de que el método **generaliza**.

Mismo filtro, otra tarea: si funciona acá, sirve en general.

Cuatro estrategias de filtrado evaluadas

Filtro	Criterio	Parámetros
None (control)	Sin filtrado, todas las muestras LLM.	—
LOF	Densidad local; conserva inliers de `decision_function` (sklearn).	k = 20 vecinos[†]; umbral > 0
Cascade L1–4	Serie de filtros geométricos acumulados. Top-N por media geométrica.	niveles L $\in \{1, 2, 3, 4\}$
Combined	LOF **Y** similitud coseno.	$\tau_{\cos} = 0{,}5$

Cascada de filtros: niveles 1–4

Score compuesto (media geométrica de L niveles activos):

$$s(x) = \left(\prod_{i=1}^{L} s_i(x) \right)^{1/L}$$

Nivel 1 ⋆ Distancia euclidiana al **ancla más cercana** (real más próximo):

$$d(x, a^*) = \min_{a \in \mathcal{A}} \|x - a\|_2 \longrightarrow s_1 = 1 - \frac{d}{\max d}$$

Nivel 2 Similitud coseno al ancla:

$$\cos(x, a) = \frac{x \cdot a}{\|x\| \, \|a\|} \longrightarrow s_2 = \mathrm{clip}(\cos, 0, 1)$$

Nivel 3 Pureza KNN ($k = 10$ misma clase); $\bar{d}_k$ = dist. media:

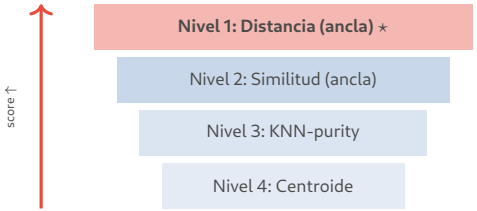
$$\bar{d}_k \longrightarrow s_3 = \frac{1/(1 + \bar{d}_k)}{\max 1/(1 + \bar{d}_k)}$$

Nivel 4 Distancia al **centroide** c:

$$d(x, c) = \|x - c\|_2 \longrightarrow s_4 = 1 - \frac{d}{\max d}$$

La flecha normaliza e invierte cada métrica a $s_i \in [0, 1]$ (máx sobre candidatos).

L2-normalización ⇒ niveles 1 y 2 dan el mismo ranking.



Selecciona top-N por ranking,

sin umbral binario.

Ponderación suave: la 2.ª contribución

En vez de **aceptar / rechazar**, asignamos un peso a cada muestra.

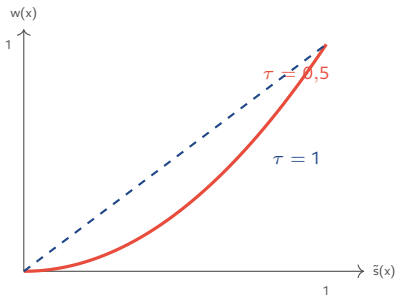
Eq. 1 — Normalización min-max:

$$\tilde{s}(x) = \frac{s(x) - s_{mín}}{s_{máx} - s_{mín}}$$

Eq. 2 — Escalado por selectividad:

$$w(x) = máx\left(w_{mín},\ \tilde{s}(x)^{1/\tau}\right)$$

$\tau = 0{,}5$, $w_{mín} = 0$, datos reales $\Rightarrow$ peso 1.



Score normalizado

$\tau < 1$ amplifica diferencias entre buenas y malas.

Las muestras se entregan al clasificador como `sample_weight`.

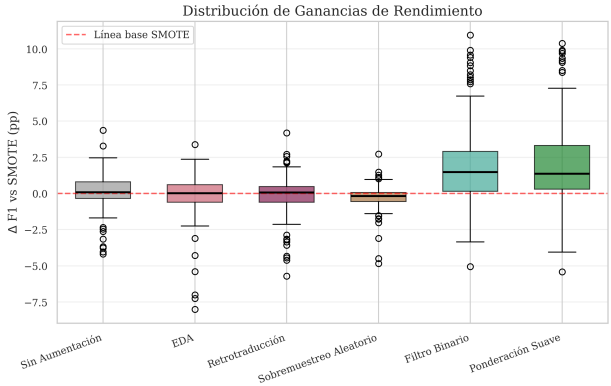
6.
Resultados

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

Resultado principal

+2,25 pp
vs. SMOTE
(macro F1)

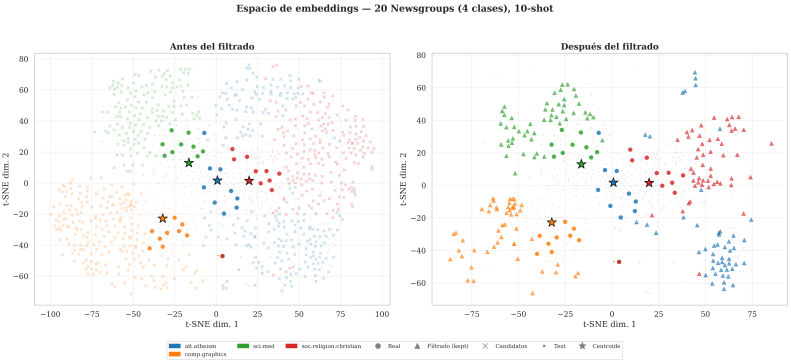
- $p < 0.0001$ [Bonferroni]
- d de Cohen $= 0.74$ [efecto medio-grande]
- Win-rate $= 83.8\,\%$
- $N = 3.675$ configuraciones
- Clasificación absoluta: **73.49 %** vs. 71.24 %



Distribución de ΔF1 por método.

El método propuesto supera todas las líneas base, incluidas las modernas.

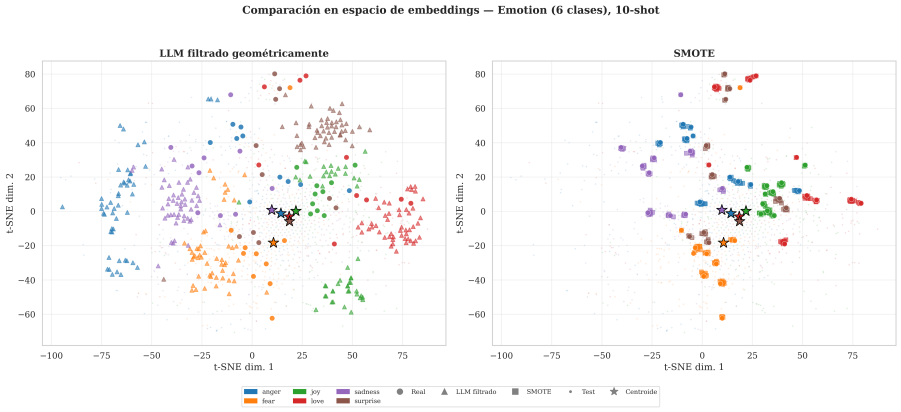
Visualización t-SNE: el filtro en acción — antes vs después



`20newsgroups` (4 clases, 10-shot). **Izq.:** candidatas del LLM (triángulos tenues). **Der.:** sólo las que pasaron el filtro. Círculos = reales, estrellas = centroides.

El filtro descarta candidatas en zonas de cruce o periféricas; conserva las que refuerzan el cluster.

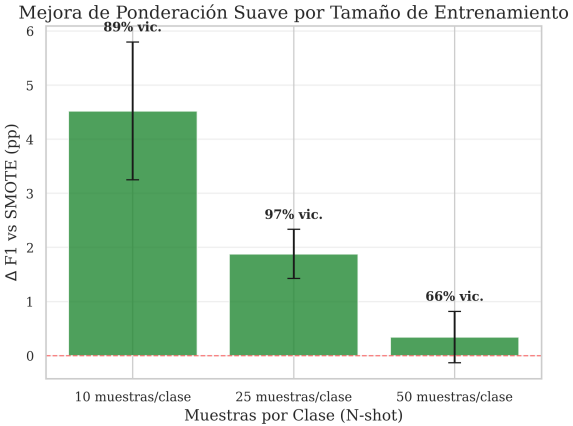
Visualización t-SNE: SMOTE vs filtro



Dataset `emotion` (6 clases, 10-shot): triángulos = LLM filtrado, cuadrados = SMOTE.

El filtro rechaza candidatas en zonas de cruce y aporta diversidad real.

Retornos decrecientes con más datos



ΔF1 por valor de n-shot.

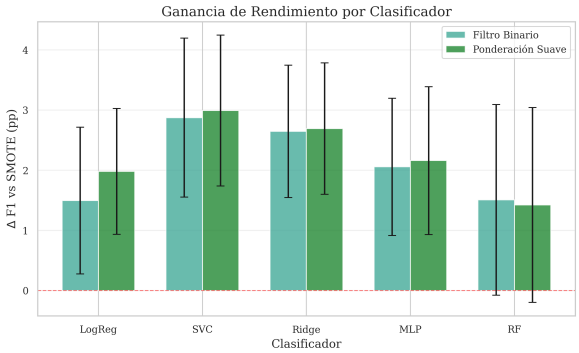
El método brilla donde más se necesita:

n-shot	ΔF1	d
10	+4.89	1.48
25	+2.17	1.57
50	+0.76	0.80

Cuanto menos datos hay, más útil es el filtrado.

Caso de uso ideal: 10 ejemplos por clase.

Beneficios de la ponderación suave por clasificador



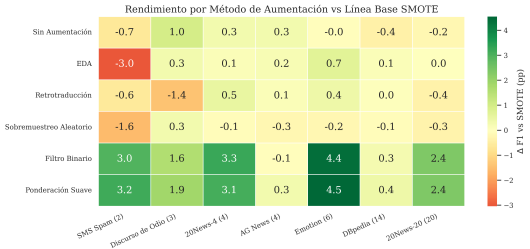
Clasificador	ΔF1	d	p
SVC (lineal)	+2.99	1.02	< 0,001
Ridge	+2.69	1.04	< 0,001
MLP	+2.16	0.73	< 0,001
Logistic Regression	+1.98	0.82	< 0,001
Random Forest†	+1.42	0.31	n.s.

p-valor con t-pareada y Bonferroni.

† RF con otra configuración (boosting/XGBoost) sí aprovecha el peso suave (XGBoost +9.10pp).

Modelos lineales explotan mejor la señal geométrica.

¿Cuándo conviene aplicarlo? Por número de clases



Sweet spot: pocas clases.

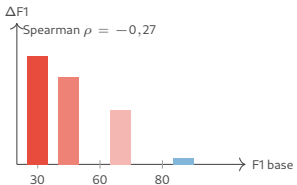
# Clases (dataset)	ΔF1
6 (`trec6, emotion`)	+5.42
4 (`20news, ag`)	+2.80
2 (binarios)	+3.18
77 (`banking77`)	−0.99
150 (`clinc150`)	+0.01

Spearman $\rho = -0.260$, p $= 0.003$.

Saturación con $\geq$ 77 clases: el espacio se fragmenta.

Resumen por dificultad: la mejora escala con el déficit

Dificultad (F1 base)	N clases	Δ medio
Difícil (<30 %)	4	**+10.44**
Media (30–60 %)	4	+8.45
Fácil (60–80 %)	4	+5.40
Muy fácil (>80 %)	1	+0.55



Lectura geométrica:

- Las clases difíciles ocupan regiones de solapamiento o frontera con otras clases.
- Las muestras sintéticas filtradas **densifican la región correcta** de la clase.
- Eso refuerza la frontera de decisión donde el clasificador tenía mayor incertidumbre.

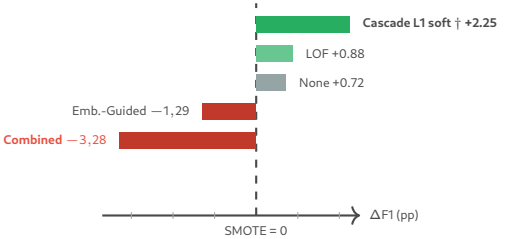
Implicancia práctica

Cuanto peor parte una clase, mayor el espacio donde el filtrado geométrico puede empujar el F1 hacia arriba. El método es **auto-correctivo** en el caso más útil.

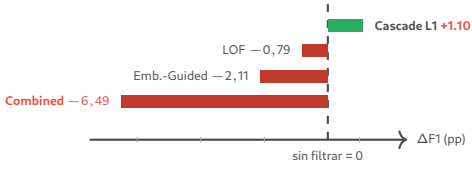
Mayor déficit inicial ⇒ mayor ganancia: el filtrado va a buscarlas.

Hallazgo contra-intuitivo: simple > complejo

ΔF1 vs. SMOTE [exp. principal, 3.675 configs]



ΔF1 vs. LLM sin filtrar [pareado, n=60]



Solo cascade L1 supera al LLM sin filtrar; los multi-criterio lo **empeoran**.

Insight clave

Filtros multi-criterio **degradan** el rendimiento.

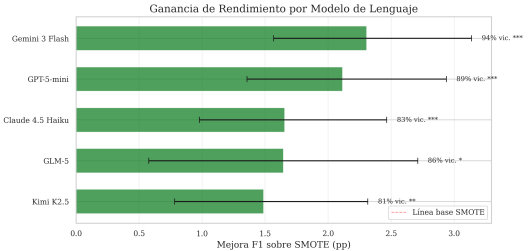
- Distancia y similitud son redundantes en L2-norm.
- LOF inestable con n = 10.
- Restricciones conjuntivas matan la diversidad.

Ablación cascada [n=60 configs]

Nivel	ΔF1 vs SMOTE	Win-rate
L0 (sin filtro)	−0.518	50.0 %
L1 ⋆ (distancia)	**+1.942**	**91.7 %**
L2 (+ similitud)	+1.942	91.7 %
L3 (+ KNN)	+1.950	91.7 %
L4 (+ centroide)	+1.923	91.7 %

Misma robustez (91.7 %) en L1–L4 ⇒ los +0.008 pp

Robustez: ¿depende del LLM o del embedding?



5 LLMs probados

Gemini, GPT-5, Claude, Kimi, GLM

ΔF1 entre $+1.49$ y $+2.31$ pp

Friedman $p = 0.086$ [no significativo]

4 modelos de embeddings

Modelo	ΔF1
mpnet-base-v2 (768d)	$+2.74$
bge-large (1024d)	$+2.04$
bge-small (384d)	$+2.00$
e5-large (1024d)	$+0.97$

90.5 % acuerdo entre modelos.

El método es agnóstico al LLM y al modelo de embeddings.

Generalización a NER: el filtro funciona en otra tarea

Reconocimiento de entidades nombradas:

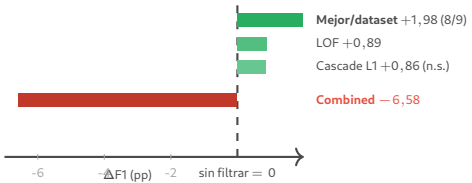
- 3 corpus: `MultiNERD`, `WikiANN`, `Few-NERD`
- Adaptación: clase = tipo de entidad dominante por oración.
- **Filtros sin modificación.**

+9.61 pp

vs. sin aumentar · Cascade L1

(p = 0,0003, d = 2,05; 10-shot +15,15)

ΔF1 vs. LLM sin filtrar [pareado, 9 datasets]



El dato LLM hace el grueso; el filtro lo **refina** (y el AND **sobre-filtra**).

La calidad geométrica es agnóstica a la tarea.

7.
Discusión y aplicaciones

1. Introducción
2. Problema
3. Estado del arte
4. Hipótesis
5. Metodología
6. Resultados
7. Discusión

Contribuciones de la tesis

Metodológica

- Sistema de filtros geométricos multi-nivel.
- Ponderación suave por score normalizado.
- Desacoplado del LLM.

Empírica

- +2.25 pp vs. SMOTE ($p < 0.0001$).
- +1.10 pp vs. LLM sin filtrar [cascade L1, $p = 0{,}017$, pareado].
- Supera a 9 baselines incluidos T5 y BERT.
- Robusto a 5 LLMs y 4 embeddings.

Generalización

- Beneficio en clases difíciles: +10.44 pp.
- NER: +9.61 pp sin modificar nada.
- Hallazgo: simple $>$ complejo.

Insight central: la información geométrica del espacio de embeddings captura calidad de manera agnóstica a la tarea, siempre que el filtro sea simple.

¿Cuándo y cómo usarlo? Utilidad práctica

Receta de aplicación:

1. Tienes 10–25 ejemplos por clase.
2. Tienes 4–10 clases.
3. Generas $3\times$ candidatas con un LLM.
4. Filtras con cascade nivel 1 (distancia).
5. Ponderas suavemente y entrenas un clasificador **lineal**.

Costo real

$\sim$ 10–50 llamadas LLM por clase.
A precios actuales: **centavos por dataset**.

Posibles casos de aplicación:

- Detección de discurso de odio [hate_speech: +13 pp en clase rara].
- Clasificación de emociones en texto [emotion: +4.55 pp; +16.9 pp en anger].
- Categorización de noticias por tema [20newsgroups, ag_news: +5 a +15 pp].
- Clasificación de tipo de preguntas [trec6: +10.12 pp].
- Detección de spam SMS [sms_spam: $\sim$ +3 pp].

Quién no debería usarlo:

- Más de 50 ejemplos por clase: SMOTE basta.
- $\geq$ 77 clases: el espacio se fragmenta.
- Modelos de árbol como única opción.

Limitaciones, trabajo futuro y cierre

Limitaciones honestas

- Evaluación solo en **inglés**.
- Caching reduce varianza [$+1.79$pp persistente en stochastic clf; sube a $\approx +$ 4pp con XGBoost].
- Pierde efectividad con ≥ 77 clases.
- LLMs comerciales filtran contenido sensible (hate-speech).

Trabajo futuro

- Datasets **multilingües**.
- LLMs locales (Llama, Mistral) sin filtros de contenido.
- Parámetro de ponderación suave T **adaptativo por clase**.
- Otras tareas: QA, resumen, generación de preguntas.

Gracias.

Preguntas, comentarios, contraejemplos: bienvenidos.

Referencias i

Sobremuestreo geométrico clásico

- Chawla, N. V., Bowyer, K. W., Hall, L. O. & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. J. Artif. Intell. Res., 16, 321–357.
- He, H., Bai, Y., Garcia, E. A. & Li, S. (2008). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. IEEE IJCNN, 1322–1328.

Aumentación textual ligera

- Wei, J. & Zou, K. (2019). EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks. EMNLP-IJCNLP, 6382–6388.
- Sennrich, R., Haddow, B. & Birch, A. (2016). Improving Neural Machine Translation Models with Monolingual Data. ACL.
- Edunov, S., Ott, M., Auli, M. & Grangier, D. (2018). Understanding Back-Translation at Scale. EMNLP, 489–500.
- Kobayashi, S. (2018). Contextual Augmentation: Data Augmentation by Words with Paradigmatic Relations. NAACL-HLT, 452–457.
- Wu, X., Lv, S., Zang, L., Han, J. & Hu, S. (2019). Conditional BERT Contextual Augmentation. ICCS, 84–95.

Aumentación en espacios de embeddings

- Zhang, H., Cisse, M., Dauphin, Y. N. & Lopez-Paz, D. (2018). mixup: Beyond Empirical Risk Minimization. ICLR.
- Chen, J., Yang, Z. & Yang, D. (2020). MixText: Linguistically-Informed Interpolation of Hidden Space for Semi-Supervised Text Classification. ACL, 2147–2157.
- Bayer, M., Kaufhold, M. A. & Reuter, C. (2023). A Survey on Data Augmentation for Text Classification. ACM Computing Surveys, 55(7), 1–39.

Referencias ii

Generación con LLMs

- Brown, T. et al. (2020). Language Models Are Few-Shot Learners. NeurIPS, 33, 1877–1901.
- Whitehouse, C., Choudhury, M. & Aji, A. F. (2023). LLM-powered Data Augmentation for Cross-lingual Commonsense Reasoning. ACL, 1234–1245.
- Suhaeni, C. & Yong, H.-S. (2023). Mitigating Class Imbalance in Sentiment Analysis through GPT-3-Generated Synthetic Sentences. Appl. Sci., 13, 9766. (+12,7 % acc. en minoritarias.)
- Gopali, S., Abri, F., Namin, A. S. & Jones, K. S. (2024). The Applicability of LLMs in Generating Textual Samples for Analysis of Imbalanced Datasets. IEEE Access, 12, 136451–136465.
- Cegin, J., Simko, J. & Brusilovsky, P. (2025). LLMs vs Established Text Augmentation Techniques for Classification: When do the Benefits Outweigh the Costs? NAACL, 10543–10570.
- Springer, J. M., Devoto, C., Tosheva, M. I. & Ramesh, P. (2025). On the Influence of Synthetic Data for Few-Shot Text Classification. Findings ACL, 8249–8268.

Fine-tuning de LLMs para generación sintética

- Cloutier, N. A. & Japkowicz, N. (2023). Fine-tuned generative LLM oversampling can improve performance over traditional techniques on multiclass imbalanced text classification. IEEE Big Data, 5181–5186.
- Nguyen, D. et al. (2024). Large Language Models for Imbalanced Classification: Diversity makes the difference (ImbLLM). arXiv:2510.09783.
- Sundararajan, N. T. & Kumarapathirage, A. (2023). SentiGEN: A Sentiment-Aware Generative Model for Text Data Augmentation. Findings EMNLP, 9138–9149.

Referencias iii

Filtrado, anomalías y selección de datos

- Breunig, M. M., Kriegel, H.-P., Ng, R. T. & Sander, J. (2000). LOF: Identifying Density-Based Local Outliers. ACM SIGMOD, 93–104.
- Robinson, M., Dey, S. & Chiang, T. (2025). Token Embeddings Violate the Manifold Hypothesis. NeurIPS.
- Tessari, F. & Hogan, N. (2024). Surpassing Cosine Similarity for Multidimensional Comparisons: Dimension Insensitive Euclidean Metric. arXiv:2407.08623.
- Du, Y. et al. (2025). Disentangling the Roles of Representation and Selection in Data Pruning. ACL, 16791–16809.
- Yu, T., Zhuang, Y., Zhang, Z., Socher, R. & Xiong, C. (2025). R.I.P.: Better Models by Survival of the Fittest Data. ICML.
- Liu, Z., Tang, Y., Wang, H. & Li, M. (2025). Residual Alignment Model: A Framework for Importance Sampling in Data Augmentation. NeurIPS.
- Tokdar, S. T. & Kass, R. E. (2010). Importance Sampling: A Review. WIREs Comput. Stat., 2(1), 54–60.

NER y extensiones

- Zhou, R. et al. (2022). MELM: Data Augmentation with Masked Entity Language Modeling for Low-Resource NER. ACL, 2251–2262.

Manifold hypothesis y embeddings

- Bengio, Y., Courville, A. & Vincent, P. (2013). Representation Learning: A Review and New Perspectives. IEEE TPAMI, 35(8), 1798–1828.
- Fefferman, C., Mitter, S. & Narayanan, H. (2016). Testing the Manifold Hypothesis. J. Amer. Math. Soc., 29(4), 983–1049.
- Muennighoff, N., Tazi, N., Magne, L. & Reimers, N. (2023). MTEB: Massive Text Embedding Benchmark. EACL, 2014–2037.

Backup B0a: ¿Por qué este protocolo estadístico?

Cada herramienta responde una pregunta **distinta** sobre el efecto:

Herramienta	Pregunta	Justificación
t-pareada	¿La diferencia es real?	Configs emparejadas por dataset $\times$ clasificador $\times$ semilla: elimina varianza entre configs.
Bonferroni	¿Y con muchos tests?	7 métodos vs. SMOTE $\Rightarrow \alpha = 0.05/7$. Conservador, blindado contra falsos positivos.
Bootstrap CI 95 %	¿Qué rango es plausible?	No asume normalidad. 10 000 remuestreos: percentiles 2.5 y 97.5 estables al 3er decimal.
d de Cohen	¿Qué tan grande es?	Separa significancia **estadística** de **práctica**: con n=525, p puede ser bajo aunque Δ sea trivial.
Win-rate	¿Qué tan consistente?	Media $+2.25$pp con 50 % win = volátil. Con 82 % win = robusto.

Los **4 apuntan** en la misma dirección $\Rightarrow$ conclusión difícil de atacar.

Backup B0b: d de Cohen — interpretación

Fórmula (pareada):

$$d = \frac{\overline{\Delta F1}}{SD(\Delta F1)}$$

Convenciones (Cohen 1988):

d	Magnitud	Interpretación
0.2	pequeño	apenas distinguible
0.5	medio	visible a ojo
0.8	grande	claramente diferente
1.0+	muy grande	casi sin solapamiento

Nuestros valores

Configuración	d	Magnitud
Global (soft-weighted)	0.74	medio-grande
SVC (lineal)	1.00	muy grande
Ridge	1.03	muy grande
MLP	0.73	medio-grande
LogReg	0.82	grande
emotion 6-clases	1.24	muy grande
20news 10-shot	0.95	muy grande

Con $d = 0{,}74$ y $p < 0{,}0001$: la diferencia es **real** y **práctica**.

Backup B0c: Robustez no-paramétrica (Wilcoxon)

La t pareada supone normalidad (aprox.) de las diferencias. El **Wilcoxon de rangos con signo** no supone nada sobre la distribución: si ambos coinciden, la conclusión **no depende del supuesto**.

Resultado principal (N=525: 21 datasets × 5 clf × 5 semillas)

vs. SMOTE	Δpp	win	p (t)	p (Wilcoxon)
soft_weighted	+2.25	82 %	1.5×10^{-49}	4.3×10^{-54}
binary_filter	+2.11	78 %	5.6×10^{-44}	6.9×10^{-46}

Por n-shot (Wilcoxon, soft_weighted)

n-shot	Δpp	p (Wilcoxon)	d
10	+4.52	6.3×10^{-25}	1.16
25	+1.88	2.0×10^{-26}	1.16
50	+0.34	1.1×10^{-5}	0.22

Implicancia

Wilcoxon coincide con la t y da p **aún menor**. El win-rate (82 %) es la prueba de signos asociada. En 50-shot sigue significativo pero con efecto **pequeño** (d=0,22) — coherente con que el método es para **muy bajo recurso**.

Backup B2: Asimetría de semillas y caching

Observación: ≈80 % de las combinaciones LLM × clasificador determinista tienen **varianza cero** entre semillas (la generación LLM está cacheada ⇒ los lineales dan F1 idéntico).

Validación 1 — subset estocástico (exp. principal, MLP+RF, varianza genuina):

- ΔF1 $= +1.79$ pp, p $= 0.0015$, d $= 0.52$.
- Wilcoxon p $= 7 \times 10^{-13}$. Significancia se mantiene.

Validación 2 — re-runs RF/XGB (`tuning_v2`, clasificadores estocásticos corridos de cero):

re-run (var. real)	Δpp vs SMOTE	p (Wilcoxon)
Random Forest	$+1.50$	0.012
XGBoost	$+9.10$	$< 10^{-9}$

Solo 22 % de varianza cero (vs ≈80 % cacheado): la varianza cross-seed es **real**.

Implicancia

Los CIs del exp. principal subestiman la varianza, pero las pruebas t pareadas **siguen válidas** (mismo split por par). Y donde la varianza es **genuina** (re-runs), el efecto no solo sobrevive: **crece** (XGBoost $+9.10$).

Si ese subset usa un árbol que sí pondera (prom. MLP $+2.16$ / RF $+1.50$ / XGB $+9.10$): de $+1.79$ a $\approx$ **$+4.3$ pp**.

Backup B0d$_3$: ¿Filtro binario o peso continuo? (Random Forest)

Las formas de usar las muestras del LLM, en Random Forest (ΔF1 macro vs SMOTE):

Modo	Δ vs SMOTE	
`no_augmentation` (sin aumentar)	−0,70pp	↓ peor que SMOTE
`llm_unfiltered` (sin filtro)	+1,39pp	
`binary_filter` (keep/reject)	**+1,89** pp	↑ el mejor
`soft_weighted` (peso continuo)	+1,50pp	↓ no mejora sobre binario

Re-ejecución `tuning_v2` (clasificador fijo, varianza real, con control `llm_unfiltered`).

En RF el aporte está en **seleccionar** (binario +1,89); **ponderar no agrega** (paso −0,39pp, n.s.): el bagging usa árboles profundos que memorizan sin importar el `sample_weight`. Opuesto a XGBoost (B0d$_4$): bagging → selección, boosting → ponderación.

Backup B0d$_4$: ¿Filtro binario o peso continuo? (XGBoost)

Las formas de usar las muestras del LLM, en XGBoost (ΔF1 macro vs SMOTE):

Modo	Δ vs SMOTE	
`no_augmentation` (sin aumentar)	−0,98pp	↓ peor que SMOTE
`llm_unfiltered` (sin filtro)	+7,96pp	
`binary_filter` (keep/reject)	+7,52pp	↓ peor que sin filtro
`soft_weighted` (peso continuo)	**+9,10** pp	↑ el mejor

Seleccionar/descartar (binario) **no aporta**; **ponderar por el score sí**. Usá el filtro geométrico como **peso continuo**, no como keep/reject.

Backup B0d$_5$: Desglose — ¿seleccionar o ponderar?

Del +2.25pp de `soft_weighted` vs SMOTE: ¿cuánto es **seleccionar** (filtro binario) y cuánto el **paso** de **ponderar** (peso continuo)?

Por familia de clasificador (ΔF1 vs SMOTE):

Familia	Binario (selección)	Suave (completo)	Paso suave (suave−bin.)
No-árbol [a]	+2.27	+2.46	+0.19
Bagging (RF) [a]	+1.51	+1.42	−0.09
Boosting (XGB) [b]	+7.52	+9.10	**+1.59*****

[a] exp. principal (No-árbol = LR/SVC/Ridge/MLP; y RF). [b] `tuning_v2`, clf. fijo. *** d=0,92.

Global — los 5 clf (N=525):

Modo	Δpp	win	d
`binary_filter` (selección)	+2.11	78 %	0.69
`soft_weighted` (completo)	+2.25	82 %	0.74
paso de ponderar	+0.13	54 %	n.s.

Ponderación suave a profundidad

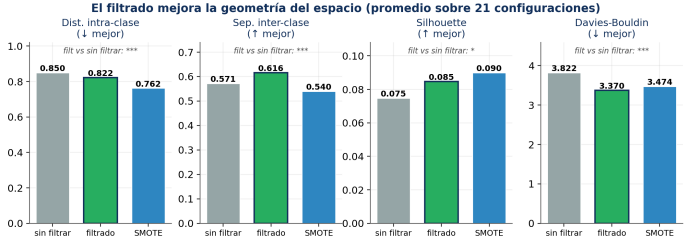
El **paso** de ponderar rinde distinto según el clasificador:

- **Lineal/MLP:** la selección ya quitó los outliers del margen ⇒ ponderar agrega poco.
- **Boosting:** cada árbol corrige el error **ponderado** del previo ⇒ el peso sí se explota (en XGB el paso binario es nulo, −0.44 n.s.).
- **Bagging:** árboles profundos memorizan sin importar el peso ⇒ ningún paso ayuda.

Por n-shot el paso ayuda más en bajo recurso: 10 → +0.36 (66 %), 25 → +0.15, 50 → −0.11.

La slide 20 muestra que el **método completo** gana en todo clasificador; este desglose muestra **por qué**: los lineales explotan la **selección**, el boosting la **ponderación**. La misma señal geométrica entra por canales distintos ⇒ H1b (peso suave) es condicional al clasificador.

Backup B6: ¿El filtro mejora la geometría? (validación intrínseca)



Filtrado mejora vs. sin filtrar

- Dist. intra-clase $0.850 \rightarrow 0.822$ [$p < 0{,}001$]: clústeres más compactos.
- Sep. inter-clase $0.571 \rightarrow 0.616$ [$p < 0{,}001$]: clases más separadas.
- Silhouette $0.075 \rightarrow 0.085$ [$p = 0{,}029$]; Davies-B. $3.822 \rightarrow 3.370$.

El matiz de SMOTE

SMOTE es **más compacto** (intra 0.762, silhouette 0.090) porque **interpola** dentro del convex hull. Pero tiene la **peor separación** inter-clase (0.540) y peor Davies-Bouldin (3.474) que el filtrado.

El filtro **cierra el lazo causal**: mejora la geometría intrínseca, no solo el F1. La compacidad artificial de SMOTE **no se traduce** en mejor clasificación; la **diversidad + separación** del LLM filtrado sí.

Backup B0f: Comparación completa de filtros geométricos

Filtro	ΔF1	Win	Selección	LLM	Notas
cascade L1 + top-N	+3.66	84.5 %	ranking top-N	3×	**Ganador**
cascade L2 (top-N)	+3.60	83.9 %	ranking top-N	3×	Similar a L1
cascade L4 (top-N)	+3.60	83.3 %	ranking top-N	3×	Redundante
cascade L1 (binary)	+1.94	91.7 %	ranking top-N	3×	Sin pesos suaves
cascade L3 (binary)	+1.95	91.7 %	ranking top-N	3×	+ KNN, no ayuda
LOF (k=5, θ=−0.5)	+2.43	90.0 %	umbral (relajado)	6.2×	Segundo mejor
LOF (k=20, θ=0)	+0.05	67.5 %	umbral (estricto)	8×	Default sklearn falla
none (control)	+0.72	62.9 %	sin filtrar	3×	LLM directo
embedding_guided	−1.29	55.0 %	cobertura greedy	–	Coverage-only falla
combined (LOF ∧ sim)	−3.28	52.5 %	AND, ∼23 % pasa	32×	Demasiado restrictivo

Selección: ranking top-N toma los N mejores del pool 3× generado (filtrado por ranking, no por umbral). Umbral: rechaza candidatos bajo el score; el % indica cuántos pasan. "LLM" = candidatos por muestra requerida.

Tres principios confirmados: (1) Ranking top-N $>$ umbral duro. (2) Distancia simple $\geq$ múltiples criterios. (3) Operador **AND** destruye la señal.

Backup B0e: Elección de hiperparámetros

LOF: `n_neighbors` y `threshold`

Param	k = 5	k = 10	k = 20
ΔF1 vs. SMOTE	+2,43	+0,16	+0,05
Win-rate	90.0 %	70.0 %	67.5 %

Default sklearn k=20 falla en 10-shot (no hay 20 vecinos por clase). k=5 es el sweet spot.

LOF: `threshold`

Threshold	ΔF1	Win-rate
−0,50	+1,29	80.0 %
−0,30	+0,06	68.3 %
0,00 (default sklearn)	+0,88	75.6 %

θ=0 es la frontera matemática inlier/outlier (*`decision_function`*).

Cascade: `k_neighbors` para KNN purity

- Fijado k=10 (compromiso 10/25/50-shot).
- L1 (sin KNN) no es sensible a este parámetro.

Combined: `sim_threshold`

θ_{sim}	ΔF1	Win-rate
0,3	−0,68	55.0 %
0,5	−5,71	41.7 %
0,7	−1,10	71.7 %

Cualquier $\theta_{sim} > 0$ combinado con AND $\Rightarrow$ catastrófico.

Top-N: peso mínimo y selectividad

- Normalización: `minmax` (vs. rank, z-score).
- Selectividad τ=0,5 (peaked, $w \propto s^{1/\tau}$).
- Peso mínimo $w_{mín}$: **poco impacto** ($\pm$0,05pp).

Hiperparámetros del cascade son **poco sensibles**; los de LOF y combined son **críticos**.

Backup B0g: Hiperparámetros LLM y ponderación suave

LLM (Gemini 3 Flash)

Param	Valor	Justificación
`temperature`	0,8	Diversidad / coherencia
`top_p`, `top_k`	default	SDK Gemini
`max_tokens`	4000	$\sim$ 50 muestras/llamada
back-trans. T	0,3	EN $\leftrightarrow$ ES conservador

Ablation `temperature` (cascade L1):

T	sms_spam	20news
0,5	$-0{,}07$	$+5{,}04$
0,7	$+1{,}44$	$+4{,}16$
0,8	$-1{,}06$	$+6{,}04$
0,9	$-0{,}18$	$+6{,}25$
1,0	$+4{,}95$	$+7{,}44$

No monótono. T=0,8 por convención.

Ponderación suave (top-N)

Param	Valor	Comentario
τ ponder.	0,5	$w \propto s^{1/\tau}$, peaked
normalización	minmax	frente a rank, sigmoid
$w_{mín}$	0,0	sensibilidad $< 0{,}2$pp
candidatos	$3\times$ target	margen top-N

Sweep Phase 2 (468 configs):

Param	ΔF1	Win
$\tau = 0{,}5$	$+4{,}10$	88.3 %
$\tau = 1{,}0$	$+3{,}67$	84.0 %
$\tau = 2{,}0$	$+3{,}41$	83.3 %
norm. minmax	$+3{,}98$	89.6 %
norm. rank	$+3{,}56$	83.3 %

τ=0,5 **peaked** amplifica diferencias entre muestras buenas y mediocres $\Rightarrow$ máxima ganancia.

Backup B0i: Hiperparámetros de los clasificadores

Experimento principal — 5 clasificadores

Defaults sklearn / PyTorch, sin tuning por dataset:

Modelo	Hiperparámetros
Logistic Reg.	L2, C=1.0, `lbfgs`, `max_iter=1000`
SVC	`kernel=linear`, C=1.0
Ridge	α=1.0
Random Forest	`n_estimators=100`, `max_depth=None`, `class_weight=None`, `max_features=sqrt`
MLP *	1 capa oculta = 128, ReLU, `dropout=0.3`, Adam lr= 10^{-3}, `weight_decay`= 10^{-4}, cosine, 300 épocas, batch 16

* implementación propia (PyTorch); pérdida CE con `sample_weight`. Todos: `random_state` por semilla; muestras reales ⇒ w=1.

Análisis de árboles (RF / XGBoost)

Fijos (referencia):

- RF = el del principal (`n_estimators=100`).
- XGB: `n_est=100`, `depth=6`, lr= 0.1, `subsample=colsample=1.0`, `hist`.

Selección por CV — `StratifiedKFold(3)` sobre **train** (sin fuga al test):

	Grilla (# combos)
RF cv	`n_est=300`; `depth`∈ {6, 10, 15}; `min_leaf`∈ {1, 3}; `cw`∈ {None,bal.} (12)
XGB cv	`depth`∈ {3, 5}; `min_child`∈ {1, 3}; `subsample`∈ {0,8, 1}; `colsample`= 0,8 (8)

Defaults a propósito en el principal: las diferencias miden el **método de aumentación**, no el ajuste del clasificador. El tuning de árboles usa CV **solo en train** (el test se toca una vez) ⇒ neutraliza el ataque "ajustaste XGBoost en el test". LLM y peso suave: B0g; umbrales de filtros: B0e.

Backup B0h: Embeddings, clasificadores y protocolo

Clasificadores (sklearn defaults)

Modelo	Hiperparámetros
LogReg	`max_iter=1000`
SVC	`kernel="linear"`
RandomForest	`n_estimators=100`
Ridge	$\alpha = 1{,}0$
MLP (custom)	128 hidden, 300 ep., lr 10^{-3}

Sin tuning por dataset $\Rightarrow$ las diferencias son del **método de aumentación**.

Embedding model (504 runs)

Modelo	Dim	ΔF1	Win
mpnet-base	768	+2,74	92.1 %
bge-large	1024	+2,04	69.8 %
bge-small	384	+2,00	76.2 %
e5-large	1024	+0,97	68.3 %

Más grande $\neq$ mejor: bge-small (384d) supera a e5-large (1024d).

Protocolo

Seeds $\{42, 123, 456, 789, 1011\}$ · N-shot $\{10, 25, 50\}$ · LLM % $\{5, 25, 50, 100\}$ · macro F1 · 50 sintét./clase · cache MD5.

Hiperparámetros del **filtro** son robustos ($\pm 0{,}5$pp). Los del **embedding** y **clasificador** dominan la varianza.

Backup B5: Comparación con baselines modernos

1.890 configuraciones, 10 métodos, 3 clasificadores:

Método	ΔF1 vs. SMOTE	p Bonf.	Win-rate
Soft-weighted (ours)	+2.61	< 0.0001	88.9 %
Binary-filter (ours)	+2.39	< 0.0001	79.4 %
No augmentation	+0.64	< 0.05	56.0 %
Embedding mixup	−0.01	n.s.	51.0 %
Contextual BERT	−0.26	n.s.	48.0 %
T5 paraphrase	−0.35	n.s.	47.0 %
Random oversample	−0.32	n.s.	47.0 %
EDA	−0.49	< 0.05	41.0 %
Back-translation	−0.50	< 0.05	40.0 %

Todas las técnicas modernas **pierden** contra SMOTE; solo el filtrado geométrico gana.

Backup B11: Las clases que más ganan — las que partían perdiendo

Dataset	Clase	SMOTE	Ours	Δ
Clases con F1 base bajo				
emotion	**anger**	27.8	44.6	**+16.9**
20news	**alt.atheism**	50.9	65.9	**+15.0**
emotion	joy	29.4	42.6	+13.3
hate_speech	**offensive**	73.3	86.3	+13.0
emotion	fear	48.8	59.0	+10.2
Clases con F1 base alto				
20news	comp.graphics	90.8	91.3	+0.5
emotion	sadness	41.3	42.3	+1.0

10-shot; 3 clasificadores lineales.

Patrón: las clases difíciles inicialmente son las que más ganan con filtrado geométrico.

- "anger" arranca en **27.8 %** (poco discriminable).
- Termina en **44.6 %**: +16,9 pp.
- "comp.graphics" parte ya en 90.8 %: poco margen, ganancia +0,5 pp.

Donde el clasificador **ya rinde bien**, el filtrado aporta poco.
Donde **tropieza**, el filtrado densifica la región correcta y refuerza la frontera.

El método actúa justo donde el modelo más lo necesita.

Backup B10: ¿Por qué estos datasets de clasificación?

Experimento principal — 7 datasets

Buscan **cobertura en tres ejes**:

- **Granularidad:** 2 → 20 clases.
- **Dominio:** spam, odio, noticias, emoción, ontología.
- **Estilo:** corto/largo, formal/informal, social/curado.

- `sms_spam` (2): NLP canónico, frase corta.
- `hate_speech_davidson` (3): dominio sensible.
- `20newsgroups`, `ag_news` (4): noticias.
- `emotion` (6): el ejemplo recurrente.
- `dbpedia14` (14): ontología Wikipedia.
- `20newsgroups_20class` (20): máxima granularidad.

Escalabilidad — 3 datasets adicionales

Responden la pregunta:

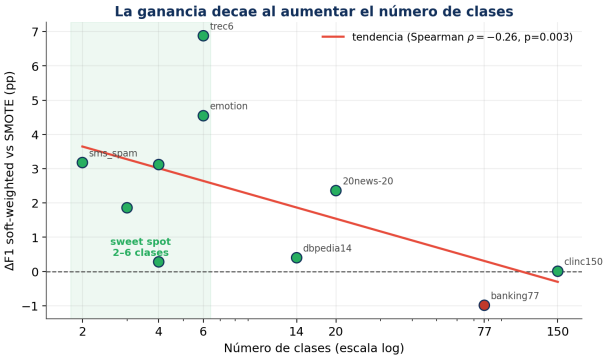
¿el filtrado sigue funcionando con muchas clases?

- `trec6` (6): preguntas TREC, mismo régimen de clases que el principal pero distinto dominio. Sirve de **control**.
- `banking77` (77): intenciones bancarias, granularidad fina con clases semánticamente cercanas.
- `clinc150` (150): intenciones de diálogo de asistentes virtuales. **Caso extremo**.

Permiten medir **degradación del filtrado** al aumentar el número de clases.

Diez datasets, dos preguntas: ¿qué tan bueno es el filtrado? ¿hasta dónde escala?

Backup B9: Escalabilidad — ¿por qué decae con muchas clases?



La ganancia cae con más clases.

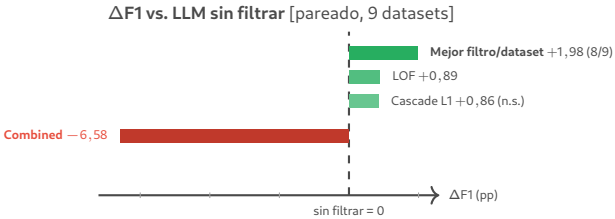
- Sweet spot **2–6 clases**: $+3$ a $+7$pp.
- dbpedia14 (14): $+0.40$; banking77 (77): -0.99; clinc150 (150): $+0.01$.
- Spearman $\rho = -0.260$ [p $=$ 0,003, n $=$ 132].

Mecanismo

Con muchas clases el espacio se **fragmenta**: todo cae "lejos" del ancla y la distancia pierde poder discriminativo. Además, clases semánticamente cercanas (banking77) se solapan.

Límite por diseño: el método es para **bajo-recurso con pocas clases** (2–6), no para taxonomías de 77+ clases (ahí convienen enfoques jerárquicos).

Backup B13: NER — ¿cuánto aporta el filtro sobre el LLM sin filtrar?



El filtro fijo agrega poco (no-sig.); el AND (combined) **sobre-filtra**.

El grueso lo aporta el dato LLM

F1 macro medio (9 datasets):

sin aumentar **23.3** $\xrightarrow{+8,4}$ LLM s/filtrar **31.7** $\xrightarrow{+0,9}$ filtrado **32.6**

Filtrado vs sin filtrar por n-shot:

n-shot	Δ (cascade L1)
10	+1,18
25	+1,89
50	−0,48

Igual que en clasificación: el filtro ayuda en bajo-recurso.

El titular de la slide principal ($\sim +9{,}6$ vs sin aumentar) combina **dato LLM** ($+8{,}4$) y **filtro**. El aporte marginal del filtro sobre el LLM sin filtrar es **modesto** ($+0{,}9$ fijo, n.s. $\rightarrow +1{,}98$ mejor-por-dataset, 8/9), pero **positivo y en otra tarea sin modificar el filtro**: la prueba más fuerte de generalización.

Backup B3: Closed-loop vs. pasada única

Hipótesis exploratoria: ¿mejora si re-prompteamos al LLM con feedback de los rechazos?

Estrategia	ΔF1 vs. SMOTE	Win-rate
Closed-loop (5 iteraciones)	+1.12	52 %
Pasada única ⋆	+2.08	80 %

¿Por qué falla closed-loop?

- Tasas de aceptación **decrecen** con iteraciones (-12.3 pp).
- Fallo dominante: `DISTANCE_OUTLIER` (77 %).
- Cuello de botella: **el LLM**, no el prompt.

Pasada única + filtro simple > ingeniería iterativa de prompt.

Backup B4: Filtrado geométrico vs. teacher

Comparación con un filtrado supervisado (teacher classifier):

Filtro	ΔF1 vs. SMOTE	p	Costo extra
Geométrico (binary)	$+2.78$	< 0.001	ninguno
Teacher (binary)	$+2.43$	< 0.001	entrenar clf adicional

El filtro geométrico es **más general** (no se especializa en separabilidad lineal) y **más eficiente** (no requiere entrenamiento extra).

Backup B12: ¿Estrategias más “inteligentes” ganan? No.

Probamos dos formas de aplicar el filtro **más sofisticadas** que “a todas las clases por igual”. Ninguna supera a la versión simple y uniforme.

1. Por etapas (curriculum)

Incorporar las candidatas del LLM **de a poco**, de las más fáciles a las más difíciles, en vez de todas juntas.

Estrategia	ΔF1 vs SMOTE
Todas a la vez (simple)	+2.61
Por etapas (la mejor)	+2.74

Ganancia de las etapas: solo +0.13pp, a costa de más complejidad.

2. Mezcla LLM/SMOTE adaptada por clase

Decidir **para cada clase** cuánto usar del LLM y cuánto de SMOTE (según distintas reglas), en vez de la misma receta para todas.

¿Cómo se decide la mezcla?	Δ	Win
Igual para todas (uniforme)	+2.33	91 %
Ajustada por clase (CV)	+0.77	36 %
Ajustada por clase (oráculo)	+0.44	29 %
Ajustada por clase (mediana)	−1.11	52 %

Cuanto más se “especializa” la mezcla por clase, **peor** (incluso el “oráculo” ideal).

Aplicar el filtro **uniforme y simple** gana. Ni el curriculum por etapas, ni la mezcla adaptada por clase, ni los filtros multi-criterio (combined −3.28), ni los niveles extra de la cascada (L2–L4 ≈ L1) **superan** la versión simple. Es el mismo hallazgo central: simple > complejo.

Backup B8: Circularidad del embedding — tres defensas independientes

Crítica anticipada: "Usás *all-mpnet* para filtrar Y como features del clasificador: el filtro favorece artificialmente su propio espacio."

Defensa	Por qué rompe el círculo	Resultado
(1) 4 embeddings distintos	filtrar+clasificar con mpnet, bge-large, bge-small, e5	$+2.74$ a $+0.97$pp; **90.5 % acuerdo**
(2) Teacher independiente	filtrar con un clasificador aparte, no el espacio geométrico	geom. $+2.78 \geq$ teacher $+2.43$
(3) NER, pipeline externo	evaluación con SpaCy + seqeval (independiente del embedding)	cascade L1 $+9.61$pp

Tres caminos **independientes** (distinto espacio, distinto filtro, distinta tarea+evaluación) dan la misma conclusión: la mejora **no** es un artefacto del embedding compartido.

Límite honesto

9.5 % de desacuerdo entre embeddings en 50-shot: posible efecto residual marginal.

Backup B1: SMOTE — validación de implementación

Crítica anticipada del comité:

> "¿Su SMOTE dummy-binary es equivalente al SMOTE estándar binary-mask?"

Validación:

- 27 comparaciones lado a lado.
- Diferencia máxima: 0.0000 pp.
- F1 idénticos en todos los datasets.

Línea base SMOTE **robusta y reproducible**.

Resumen

Implementación	F1 medio
dummy-binary	0.7124
binary-mask std.	0.7124
Diferencia	**0.0000**